Low Rank Matrix Completion and Real World Applications in Recommender Systems

STOR 512 Final Project - Joe Bowen and Minzheng Song

April 2025

1 Preface

Recommender systems enhance user engagement across digital platforms by predicting preferences and delivering personalized suggestions for music, movies, products, and more. A key method behind this is **Low-Rank Matrix Completion** (LRMC), which infers missing user-item interactions from existing data to personalize experiences at scale.

This literature review provides a comprehensive overview of LRMC as a whole and explores algorithms used in optimization processes for these types of problems.

2 Background on Recommender Systems

Recommender systems are created with the intention of predicting the favorability of a user with respect to new commodities. There are several real-world applications for this problem. If you are Netflix, for example, you want to have a strong recommender system to keep customers using your services. We can most easily view Recommender Systems as a subset of an information filtering system. In this, you have two different types of information-filtering systems [2]:

- **Content-Based Filtering**: A set of discrete features that characterize a commodity and build a user profile based on what the user has liked in the past.
- **Collaborative Filtering**: Uses the current user's past information on interests, combined with other users' past behavior, to form recommendations.
  - A fundamental assumption in Collaborative Filtering is that users who share similar ratings on a past set of items would have similar ratings for new recommendations.
- The focus for this project will be on *collaborative filtering* because it can be easily molded via a Low-Rank Matrix Completion Problem.

3 Matrix Completion for Recommender Systems

3.1 Problem Setup and Motivation

The matrix completion problem can be formalized by considering a setting where we have a list of m users $\{u_1, u_2, u_3, \ldots, u_m\}$ and n items $\{i_1, i_2, i_3, \ldots, i_n\}$. User preferences across a platform (ex. Netflix) are represented as an incomplete $m \times n$ matrix A, where each entry m_{ij} corresponds to the rating given by user u_i to item i_j. If user u_i has not rated item i_j, the entry m_{ij} is considered missing. Such a matrix is shown below:

$$A = \begin{bmatrix} m_{11} & \cdot & m_{13} & \cdot & m_{15} \\ \cdot & m_{22} & \cdot & m_{24} & \cdot \\ m_{31} & \cdot & \cdot & m_{34} & m_{35} \\ \cdot & \cdot & m_{43} & \cdot & \cdot \\ m_{51} & m_{52} & \cdot & \cdot & \cdot \end{bmatrix}$$

- One key difficulty in working with matrices in this setting is that they tend to be *sparse* (i.e., containing more zero entries than nonzero ones) and therefore have infinitely many solutions to the objective. In the case of Netflix, it is very unlikely every user has seen/rated every movie or show the platform offers, the challenge remains: how can we make recommendations based on unseen data?

3.2 Objective

Let $\Omega \subseteq \{1, \ldots, m\} \times \{1, \ldots, n\}$ be the set of index positions where ratings are observed in the matrix A. For our objective, we define a masking operator $\Omega(\cdot)$ that, when applied to a randomly initialized matrix $X \in \mathbb{R}^{m \times n}$, retains only the entries at the index positions in Ω and sets all other entries to zero [2,3].

$$[\Omega(X)]_{ij} = \begin{cases} X_{ij}, & \text{if } (i,j) \in \Omega \\ 0, & \text{otherwise} \end{cases}$$

We also define a "true" underlying matrix, denoted as X^*, which is typically unknown in practice, especially when working with recommender systems. The observed matrix A is equivalent to applying the masking operator to X^*, i.e., $A = \Omega(X^*)$. Using these definitions, we can now express our objective function for recovering the missing entries:

$$\min_{X \in \mathbb{R}^{m \times n}} \quad \frac{1}{2} \|\Omega(X) - \Omega(X^*)\|_F^2 \tag{1}$$

- The objective above aims to reconstruct the solution matrix X, from random initialization, such that it recovers the known values of A, while completing missing entries in a way that provides an approximation of the true X^*.

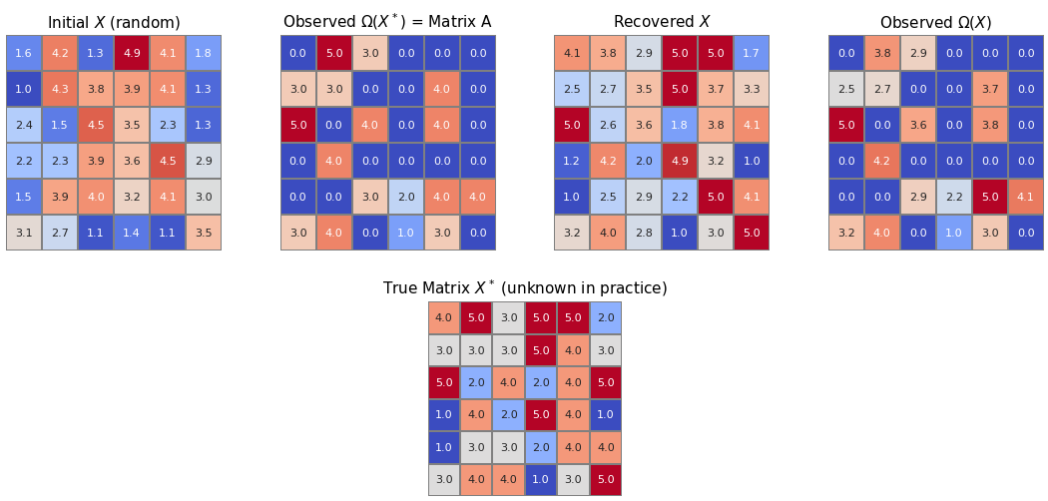

Figure 1: Visualization of Objective

3.3 Defining the Low-Rank Assumption

It is a common belief that in something such as a movie recommender system, only a few latent factors actually influence how much a user likes an item. Such factors could represent hidden attributes, such as mood or actor affinity, that are not directly observed, but may influence user decisions. This allows us to introduce the ***"low-rank"*** assumption onto our objective when solving for our solution matrix.

$$\min_{X \in \mathbb{R}^{m \times n}} \quad \frac{1}{2} \|\Omega(X) - \Omega(X^*)\|_F^2 \quad \text{s.t.}\, \text{rank}(X) \leq r \tag{2}$$

Although the original matrix $A \in \mathbb{R}^{m \times n}$ is considered to have full rank, this assumption tells us the actual data that completes A is contained in a lower-dimensional subspace. We can therefore use X, constrained to this low-rank assumption to solve our objective. It is important to note that this constraint is only effective when r is known or easily findable (e.g. Image Completion) and that this constraint is only effective in non-convex scenarios.

3.3.1 Determining Proper Rank via Rank Optimization

Methods for determining a "proper" rank, or r, typically vary on a case-by-case basis with either non-convex scenarios (2) or convex. There are scenarios, although rare in practice for this type of problem, where the rank r is known in advance. It is more common to solve the problem without prior rank information. The official Rank Minimization Equation is:

$$\min_X \ \text{rank}(X) \quad \text{subject to } X_{ij} = A_{ij}, \quad (i,j) \in \Omega$$

- This function is considered NP-Hard because it is **non-convex** and **combinatorial**. Think of it like asking, *"what is the smallest number of basis vectors I need to explain all observed entries?"* When all the data is linearly independent and you are working with a large matrix, this is computationally unachievable.

We can instead shift our focus to a convex problem by minimizing the nuclear norm in place of rank. The Nuclear Norm is defined as the sum of the singular values of X and yields the problem:

$$\min_X \ ||(X)||_* \quad \text{subject to } X_{ij} = A_{ij}, \quad (i,j) \in \Omega$$

- In certain instances, by relaxing the constraint of the rank function and switching to a convex operation, solving the nuclear norm minimization will exactly recover the original low-rank matrix with high probability [2].

While the above equation establishes a convex formulation for matrix completion, it does not account for noisy/incomplete data. To combat this in real-world scenarios, we can add a regularization term that penalizes deviation from known entries to the nuclear norm and combine it with our objective; this results in the problem below:

$$\min_{X \in \mathbb{R}^{m \times n}} \ \frac{1}{2} \left\| \Omega(X) - \Omega(X^*) \right\|_F^2 + \lambda \left\| X \right\|_* \tag{3}$$

λ is a pre-set hyperparameter and several algorithmic solvers can exploit the nuclear norm regularization for the benefit of optimizing the objective. Again, because r is unknown in practice, this allows us to have a different avenue to recover the true underlying matrix.

4 Computational Algorithms for LRMC

While several algorithms have been recognized as effective for low-rank matrix completion within the machine learning community, each of them has its strengths in the context of certain problems. We can view a few examples of such methods as they relate to STOR-512:

4.1 Alternating Least Squares

Alternating Least Squares (ALS), also called Alternating Minimization, is an algorithm where the goal low-rank Matrix X is represented using two latent feature matrices; the algorithm alternates between finding the best of each matrix. The main idea can be displayed via its mathematical notation:

$$X \approx UV^\top$$

where $\mathbf{U} \in \mathbb{R}^{m \times r}$ is the user latent feature matrix, $\mathbf{V} \in \mathbb{R}^{n \times r}$ is the item latent feature matrix, and the product $\mathbf{U}\mathbf{V}^\top \in \mathbb{R}^{m \times n}$ reconstructs X with the observed ratings as well as approximations for the unseen data.

The optimization objective typically takes the form:

$$\min_{U,V} \sum_{(i,j) \in \Omega} \left(X_{ij} - (UV^\top)_{ij} \right)^2 + \lambda \left(\|U\|_F^2 + \|V\|_F^2 \right) \tag{4}$$

λ is a regularization parameter to prevent overfitting. This objective ensures the approximation fits the known data while maintaining low-rank structure and regularity in the latent factors.

ALS is generally viewed as a **constrained optimization equation** because the solution matrix must satisfy the rank constraint listed in (2) in addition to a constraint that the predicted matrix must match the known entries from the original matrix A:

$$\Omega(X) = \Omega(X^*)$$

This constraint ensures that the approximation X agrees with the known entries in A, while the ALS algorithm enforces a low-rank structure by factorizing X as $UV^\top$. The optimization alternates between solving for U and V using least squares over the observed values, and implicitly satisfies the rank constraint by keeping the rank of X at most r. Notably, ALS was one of the primary techniques employed during the Netflix Prize Recommendation Challenge in 2006.

4.2 Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is an iterative optimization algorithm that updates the model parameters by taking small steps in the direction opposite to the gradient of the objective function. Unlike gradient descent, which computes the gradient using the entire dataset, SGD calculates it using a single randomly chosen data point.

In order to properly understand the way SGD works in training the objective, it is worth defining the equation [3]:

$$x^{k+1} = x^k - \alpha \nabla f_{i_k}(x^k)$$

where:

- x^k is the current estimate of the matrix entries.
- α is the learning rate (typically $\alpha = \frac{1}{L}$, where L is the Lipschitz constant of ∇f).
- $\nabla f_{i_k}(x^k)$ is the gradient of the loss function with respect to a single observed entry $i_k \in \Omega$ that is selected uniformly at random.

In the context of LRMC, SGD can be applied by iteratively selecting an observed entry $(i, j) \in \Omega$ and updating the i-th row of the latent factor matrix U and the j-th row of V [1].

$$U_i \leftarrow U_i - \alpha \left(\left((U_i V_j^\top) - X_{ij} \right) V_j + \lambda U_i \right)$$
$$V_j \leftarrow V_j - \alpha \left(\left((U_i V_j^\top) - X_{ij} \right) U_i + \lambda V_j \right)$$

where the gradient is calculated from the loss function defined in (3).

An advantage of using SGD is that it can be computationally efficient, especially for sparse matrices in this context, as it processes only one data point at each update. However, SGD can exhibit noisy convergence for the same reason of random sampling of the data point, and the learning rate α requires tuning for a good performance.

Overall, SGD is well-suited for large-scale recommender systems due to its low memory usage and efficient updates on sparse datasets. However, SGD is sensitive to the choice of learning rate and initialization, and may converge to suboptimal solutions due to the non-convexity of the factorization problem. Additionally, working with SGD in practice may cause Exploding/Vanishing gradients as a byproduct of sensitivity to hyperparameters which can be costly. Adaptive variants such as AdaGrad or Adam can help improve convergence stability. Regularization and early stopping are often necessary to prevent overfitting and ensure generalization.

4.3 Accelerated Proximal Gradient

Accelerated Proximal Gradient (APG) methods are advanced optimization algorithms designed to solve problems involving a combination of a smooth, differentiable function and a non-smooth, possibly non-differentiable function (like the nuclear norm term in LRMC). These methods achieve faster convergence than standard proximal gradient methods by incorporating momentum. The general update rule involves a proximal operator [1,3]:

$$x^{k+1} = \text{prox}_{\alpha g}\left(x^k - \alpha \nabla f(x^k)\right)$$

where:

- x^k is the current estimate.
- α is the step size (typically $\alpha = \frac{1}{L}$, where L is the Lipschitz constant of ∇f).
- $\text{prox}_{\alpha g}(z) = \arg\min_x \left\{\alpha g(x) + \frac{1}{2}\|x - z\|^2\right\}$ is the proximal operator of the non-smooth function g.

The core idea involves iteratively refining an estimate of the low-rank matrix X. For a given previous estimate Y, construct a quadratic approximation of the smooth part of the objective function at Y. Then substituting the quadratic approximation in (3) would give the reformulated minimization model where $\delta > 0$ is a proximal parameter [1]:

$$\min_X \frac{1}{2}\|X - (Y - \delta(\Omega(Y) - \Omega(X^*)))\|_F^2 + \lambda\delta\|X\|_*$$

The iterative scheme generates a sequence of matrices X^k, Y^k, and acceleration parameters t^{k+1} as follows [1]:

- Momentum Step:

$$Y^k \leftarrow X^k + \frac{t^{k-1} - 1}{t^k}(X^k - X^{k-1})$$

- Proximal Update:

$$X^k \leftarrow \text{prox}_{\frac{\lambda}{\delta}\|\cdot\|_*}\left(Y^k - \delta(\Omega(Y^k) - \Omega(X^*))\right)$$

  In the context of LRMC, the proximal operator for the nuclear norm is the Singular Value Thresholding (SVT) operator (discussed below in 4.4)

- Update Acceleration Parameter:

$$t^{k+1} \leftarrow \frac{1 + \sqrt{1 + 4(t^k)^2}}{2}$$

An advantage of using APG methods for LRMC problems is that when the problem is formulated as minimizing the squared error plus the regularization term that promotes low rank, the proximal operator can handle the non-smooth regularization term effectively. However, the initialization (a guess of the true matrix) and the choice of learning rate can largely affect the model's performance.

Overall, APG methods, including FISTA, achieve faster convergence rates than standard gradient methods, making them effective for nuclear norm minimization. APG is particularly useful for large, sparse matrices and can be combined with partial SVD to reduce computational overhead. The use of momentum helps escape shallow regions in the optimization landscape. However, careful tuning of step size and initialization is essential for robust performance.

4.4 Singular Value Thresholding

Singular Value Thresholding (SVT) is an algorithm specifically designed to solve nuclear norm minimization problems involved in LRMC problems. It operates by performing a singular value decomposition and then shrinking the singular values of the matrix at each iteration. The SVT operator is defined as [1,3]:

$$\text{SVT}_\tau(X) = U \cdot \text{diag}\left([\sigma_i - \tau]_+\right) \cdot V^\top$$

where:

- $X = U\Sigma V^\top$ is the Singular Value Decomposition (SVD) of the matrix X.
- $\Sigma = \text{diag}(\sigma_i)$ is a diagonal matrix containing the singular values σ_i of X in non-increasing order.
- $\tau > 0$ is a thresholding parameter.
- $[\sigma_i - \tau]_+ = \max(0, \sigma_i - \tau)$ is the soft-thresholding operator applied to each singular value.

The advantage of SVT-based methods is that they guarantee the low-rank structure of the solution matrix X by truncating small singular values. However, a key challenge is the computational cost. The algorithm involves computing the full SVD at each iteration, which could be computationally expensive for large matrices. At the same time, the thresholding parameter τ requires tuning to achieve a good convergence.

Overall, SVT serves as the proximal operator for nuclear norm minimization and is foundational to many convex LRMC solvers. In theory, SVT guarantees exact recovery under incoherence, which ensures that the underlying low-rank matrix does not concentrate too much information in a small number of entries in addition to sufficient sampling. While SVT ensures convergence to a low-rank solution, the need for computing full or partial SVD at each iteration can be computationally expensive. Techniques like truncated or randomized SVD are often used to mitigate this cost in large-scale settings. The best use case for SVT is likely in scenarios where the matrix is small and computing SVD's for each entry is in the realm of possibility with regard to computing power.

5 Discussion

Recommender Systems and Machine Learning play a crucial role in the foundation of generalization in practical settings such as movies, music, or e-commerce. Low-Rank Matrix Completion is an instrumental aspect of this, and several methods were examined for recovering the optimal solution based on (3).

While this literature review is rooted in early problems like the 2006 Netflix Recommendation Prize, the field has since evolved significantly. Today, multiple matrices, such as A, may capture various user behaviors, including time spent per genre, scroll-through rates, and unfinished plays, all of which may be used to make recommendations. You are no longer trying to recover a single matrix based on ratings.

5.1 Comparison of Methods and Discussion of Trade-Offs

Each of the low-rank matrix completion methods explored offers distinct advantages in the context of recommender systems.

Alternating Least Squares (ALS) is highly parallelizable and was notably used in the Netflix Prize Challenge. It is easy to implement and scales well to large datasets. However, it requires careful initialization and may converge slowly or get stuck in local minima [2].

Stochastic Gradient Descent (SGD) is lightweight and particularly effective for large, sparse matrices. It can be run in an online fashion, making it ideal for real-time systems. Nevertheless, its convergence can be noisy, and performance is sensitive to the learning rate and regularization parameters [1,2].

Accelerated Proximal Gradient (APG) methods offer faster theoretical convergence rates compared to standard gradient descent by incorporating momentum. They are well-suited for convex optimization problems involving nuclear norm regularization. However, they require good initialization and careful tuning of step size and momentum terms [1].

Singular Value Thresholding (SVT) provides solid theoretical guarantees and is proven to recover the true low-rank matrix under certain conditions like incoherence and sufficient sampling [2]. However, it is computationally expensive due to repeated singular value decompositions, making it less practical for very large-scale systems unless approximations like truncated SVD are used.

In summary, ALS and SGD are preferable for large-scale, real-time environments, while SVT and APG are more suitable when accuracy and theoretical guarantees are prioritized. The choice of algorithm depends on the trade-off between scalability, convergence behavior, and computational cost.

5.2 Challenges Observed

While Low-Rank Matrix Completion (LRMC) has demonstrated substantial success in recommender systems, several limitations and challenges persist.

First, many theoretical guarantees assume ideal conditions such as uniform random sampling and matrix incoherence. In practice, user-item interactions are often biased; popular items receive more ratings, and many users only interact with a small subset of items. These real-world deviations can undermine the performance of LRMC algorithms [2].

Second, scalability is a significant concern. Convex optimization methods like SVT and APG, although theoretically robust, are computationally intensive for high-dimensional matrices typical of modern platforms. As noted in [1], deploying these methods at scale requires approximations such as partial or randomized SVD, which introduce trade-offs between speed and accuracy.

Furthermore, determining appropriate hyperparameters—such as matrix rank r, regularization parameter λ, and learning rate α—remains non-trivial. These parameters are often dataset-specific and require extensive tuning, which adds to the computational burden.

Another emerging challenge is that modern recommender systems often involve multiple data modalities (e.g., time, location, user behavior), which cannot be effectively captured by a single matrix. Extending LRMC frameworks to handle multi-relational data or tensor completion is an active area of research [2].

Finally, as recommender systems evolve, there is a growing need for interpretable models. While low-rank approximations are compact, they are often opaque. Developing methods that balance predictive performance with interpretability is a promising but under-explored direction.

These challenges underscore the need for continued innovation in scalable, adaptive, and robust matrix completion techniques.

6 References

[1] A. Ramlatchan, M. Yang, Q. Liu, M. Li, J. Wang and Y. Li, "A survey of matrix completion methods for recommendation systems," in Big Data Mining and Analytics, vol. 1, no. 4, pp. 308-323, December 2018, doi: 10.26599/BDMA.2018.9020008. keywords: Collaboration;Mathematical model;Predictive models;Computational modeling;Matrix decomposition;Prediction algorithms;Computer science;matrix completion;collaborative filtering;recommendation systems,

[2] L. T. Nguyen, J. Kim, and B. Shim, "Low-Rank Matrix Completion: A Contemporary Survey," *arXiv preprint arXiv:1907.11705*, Jul. 2019. [Online]. Available: `https://arxiv.org/abs/1907.11705`

[3] M. J. O'Neill, "Lecture 21: Low Rank Matrix Problems," STOR 512: Optimization for Machine Learning and Neural Networks, Dept. of Statistics and Operations Research, Univ. of North Carolina at Chapel Hill, Lecture notes, Spring 2025. [Online].